Handwritten digit classification using machine learning techniques and neural networks

Sara Santos García

November 1, 2026

Abstract

This paper addresses the problem of handwritten character recognition. Several classical machine learning models (k-Nearest Neighbors and Support Vector Machines), dimensionality reduction techniques (Principal Component Analysis), a neural network with one hidden layer (Multilayer Perceptron), and a Convolutional Neural Network (CNN) are compared. The objective is to analyze the different results, optimize the neural network, and determine the need for a convolutional network as the problem increases in difficulty.

1 Problem description and datasets

The chosen problem is handwritten character recognition, as it is a classic problem in the field of computer science and the example presented in the manual. My idea is to initially use the parameters exactly as described in said manual and then make the pertinent modifications to analyze the changes in the accuracy of the different models.

For the classical machine learning models and deep neural networks, the `digits` dataset from the Scikit-Learn library was used. This dataset contains 1,797 grayscale images of 8×8 pixels.

The first step in all cases was to reshape these 64-element matrices into one-dimensional vectors. The target vector Y contains labels with digits from 0 to 9, corresponding to each of the digits to be recognized, which defines this task as a multi-class classification problem. Therefore, in the case of neural networks, we must have at least 10 output neurons and more than one layer, since a single neuron solves a binary classification problem.

The data is randomly divided into a training set and a test set, so that the algorithm or network does not memorize; it faces data it has not seen before when evaluating.

Furthermore, it was decided to include a Convolutional Neural Network to solve the same problem but with the `MNIST` dataset, whose handwritten digit images are 28×28 pixels and a priori require more advanced techniques. This dataset consists of 60,000 training images and 10,000 test images, much larger than `digits`. It was also decided to solve this problem with the previously mentioned neural network to determine if the convolutional one is truly necessary and if it represents an improvement.

2 Applied techniques and design justification

To establish a rigorous comparison between classical machine learning models and neural networks, k-Nearest Neighbors, Support Vector Machines (SVM), and Principal Component Analysis (PCA) were used, which will be compared with a two-layer neural network.

2.1 k-Nearest Neighbors (k-NN)

As an initial model, the non-parametric k-nearest neighbors algorithm was used, which assumes that samples of the same class are located in nearby regions. The hyperparameter was set to $k = 3$, a low value, to avoid crossing decision boundaries, and the Euclidean metric (the usual distance) was used.

2.2 Support Vector Machines (SVM)

SVMs are another type of classifier, in this case parametric, that seek a separation hyperplane with the maximum margin.

Two variants were evaluated to study the impact of the problem's non-linearity: a linear SVC model versus a polynomial SVC. Specifically, the use of a degree 2 polynomial kernel to transform the input space is justified, presuming that non-linear combinations better separate digits that look alike (such as 3 and 8).

2.3 Principal Component Analysis (PCA)

Since the original feature space consists of 64 dimensions, PCA is applied as an unsupervised learning algorithm for dimensionality reduction.

The preprocessing includes centering the data to a zero mean (necessary for this algorithm). It was decided to retain $k = 20$ components, opting for a balance between reducing computational cost and preserving the images' information.

Subsequently, a polynomial SVM was evaluated on this reduced space.

2.4 Neural network with 1 hidden layer or Multilayer Perceptron (MLP)

Here, the case of deep artificial neural networks is addressed using Keras. The initial architecture used was the one presented in the course manual. This consists of an input layer (64 features), a hidden layer (100 neurons with sigmoid activation), and an output layer (10 neurons). One-hot encoding was applied to the target vector and the *softmax* activation function was used at the output to interpret the predictions as a probability vector. Training was guided by minimizing the Mean Squared Error (MSE) through Mini-Batch Stochastic Gradient Descent (SGD) for 3000 epochs with a subset of 32 samples.

It was decided to make different changes to optimize the neural network's performance. First, the error function optimizer was modified to RMSprop (*Root Mean Square Propagation*). Additionally, the *epochs* were reduced since the rapid convergence of RMSprop allowed it, and different activation functions were tested for the first neuron layer, including *sigmoid, tanh, softmax*, and *ReLu*. Details are in the results and critical analysis section.

To make a direct comparison with a convolutional network, it was decided to adapt the `MNIST` dataset from the following section to this single-layer network as well.

2.5 Convolutional network

For the convolutional network example, the `MNIST` dataset of 28×28 digit images was used, because this is where this type of network shines, by adding complexity and size to the images. A convolution algorithm was used (with *ReLu* activation), followed by pooling and a flatten layer, after which a neural network with 1 hidden layer acts.

The selected hyperparameters were the *categorical crossentropy* loss function optimized by SGD (we already saw it was not optimal), *ReLu* and *softmax* network activation functions, and a batch size of 100, which gives 600 iterations per epoch.

3 Results and model comparison

The result of the k-NN algorithm is `0.97555`.

The result of the linear SVM algorithm is `0.94888`.

The result of the polynomial SVM algorithm is `0.99555`.

The result of the PCA + SVM algorithm is `0.98222`.

The result of the neural network with the mentioned hyperparameters, after 8 minutes, is:

```
Epoch 1/3000 34/34 ---------------- 1s 3ms/step - accuracy: 0.1271 - loss: 0.0955
Epoch 2/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1308 - loss: 0.0950
Epoch 3/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1317 - loss: 0.0945
Epoch 4/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1317 - loss: 0.0941
Epoch 5/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1336 - loss: 0.0937
Epoch 6/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1354 - loss: 0.0933
Epoch 7/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1391 - loss: 0.0929
Epoch 8/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1391 - loss: 0.0925
Epoch 9/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1410 - loss: 0.0922
Epoch 10/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1419 - loss: 0.0918
Epoch 11/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1456 - loss: 0.0915
Epoch 12/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1475 - loss: 0.0912
...
Epoch 3000/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.9972 - loss: 0.0025
       8/8 ---------------- 0s 7ms/step - accuracy: 0.9624 - loss: 0.0072
       [0.007162821479141712, 0.9624478220939636]
```

We can observe them briefly in the following table:

Evaluated Model	**Accuracy Rate (Test)**
k-Nearest Neighbors ($k = 3$)	$\approx 97.6\%$
SVM (Linear)	$\approx 94.9\%$
SVM (Polynomial, $d = 2$)	$\approx 99.6\%$
PCA ($k = 20$) + Polynomial SVM	$\approx 98.2\%$
Multilayer Perceptron (Keras)	$\approx 98.3\%$
Convolutional Network 28x28	$\approx 96.7\%$
Neural Network 28x28	$\approx 89\%$

Table 1: Comparison of the effectiveness of the implemented models.

3.1 Optimization of the multilayer neural network

We observe that the convergence is slow, and although an accuracy rate of almost 100% is reached during training, when evaluated, that rate drops to 96.2%. This is probably the result of overfitting and the vanishing gradient problem, to which the sigmoid function is not immune. On the other hand, using RMSprop as the optimization function, we obtain, after only 100 *epochs*:

```
Epoch 1/100 34/34 ---------------- 1s 3ms/step - accuracy: 0.3868 - loss: 0.0771
Epoch 2/100 34/34 ---------------- 0s 4ms/step - accuracy: 0.7217 - loss: 0.0524
Epoch 3/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.8451 - loss: 0.0366
Epoch 4/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9017 - loss: 0.0268
Epoch 5/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9286 - loss: 0.0205
Epoch 6/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9388 - loss: 0.0162
Epoch 7/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9536 - loss: 0.0130
Epoch 8/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9555 - loss: 0.0110
Epoch 9/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9685 - loss: 0.0093
Epoch 10/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9694 - loss: 0.0081
```

```
Epoch 11/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9740 - loss: 0.0071
Epoch 12/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9777 - loss: 0.0062
Epoch 13/100
...
Epoch 100/100 34/34 ---------------- 0s 2ms/step - accuracy: 1.0000 - loss: 9.2405e-05
       8/8 ---------------- 0s 6ms/step - accuracy: 0.9666 - loss: 0.0049
       [0.004873801954090595, 0.9666203260421753]
```

A much faster convergence is clearly observed in the first steps, although to some extent a problem of overfitting is still observed because the test accuracy rate drops compared to the training accuracy (we are still using sigmoid as activation in layer 1).

Finally, using ReLu as the activation function in the 1st layer, we obtain:

```
Epoch 1/100 34/34 ---------------- 1s 3ms/step - accuracy: 0.3312 - loss: 0.0992
Epoch 2/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.5909 - loss: 0.0586
Epoch 3/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.6948 - loss: 0.0431
Epoch 4/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.7403 - loss: 0.0371
Epoch 5/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8302 - loss: 0.0248
Epoch 6/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8590 - loss: 0.0201
Epoch 7/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8701 - loss: 0.0183
Epoch 8/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8748 - loss: 0.0171
Epoch 9/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8813 - loss: 0.0162
Epoch 10/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9221 - loss: 0.0109
Epoch 11/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9545 - loss: 0.0069
Epoch 12/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9638 - loss: 0.0055
...
Epoch 100/100 34/34 ---------------- 0s 3ms/step - accuracy: 1.0000 - loss: 3.2713e-06
       8/8 ---------------- 0s 5ms/step - accuracy: 0.9833 - loss: 0.0026
       [0.00258587091229856, 0.9833101630210876]
```

with a convergence similar to the previous case but with an accuracy rate almost 2% higher, which depending on the situation can be fundamental. This better result can be explained by ReLu's immunity to the vanishing gradient problem, although it shouldn't be of great importance in our case, since we only have one intermediate layer of neurons.

To make a final comparison, the activation function of the first layer and the *epochs* were modified almost to the limit of what convergence allows; with 10 iterations we obtain the following results:

Evaluated Function	**Accuracy Rate (Test)**
Sigmoid	$\approx 95.7\%$
Tanh	$\approx 97.1\%$
Softmax	$\approx 55.4\%$
ReLu	$\approx 95.6\%$

Table 2: Comparison of the effectiveness of the activation functions.

It is therefore concluded that in this task and with this neural network, the importance of the *loss function* optimizer is crucial. It allows us to optimize the resources to train the model and reduce the computational load. We also see that the activation function carries its weight, although it is not as crucial in the case of a two-layer neural network. And finally, we observe that comparing with the results with 100 *epochs*, the network learns very fast at the beginning and slower at the end.

3.2 Convolutional network results

After almost 4 minutes, the output of the convolutional network is:

```
Epoch 1/10 600/600 ---------------- 23s 35ms/step - accuracy: 0.8117 - loss: 0.7432
Epoch 2/10 600/600 ---------------- 19s 32ms/step - accuracy: 0.9153 - loss: 0.2930
Epoch 3/10 600/600 ---------------- 23s 35ms/step - accuracy: 0.9306 - loss: 0.2388
Epoch 4/10 600/600 ---------------- 20s 33ms/step - accuracy: 0.9412 - loss: 0.2021
Epoch 5/10 600/600 ---------------- 26s 44ms/step - accuracy: 0.9491 - loss: 0.1768
Epoch 6/10 600/600 ---------------- 42s 45ms/step - accuracy: 0.9539 - loss: 0.1569
Epoch 7/10 600/600 ---------------- 34s 32ms/step - accuracy: 0.9585 - loss: 0.1410
Epoch 8/10 600/600 ---------------- 20s 31ms/step - accuracy: 0.9626 - loss: 0.1282
Epoch 9/10 600/600 ---------------- 18s 29ms/step - accuracy: 0.9659 - loss: 0.1169
Epoch 10/10 600/600 ---------------- 22s 31ms/step - accuracy: 0.9681 - loss: 0.1079
       313/313 ---------------- 2s 5ms/step - accuracy: 0.9698 - loss: 0.0989
       [0.09888315200805664, 0.9697999954223633]
```

and it is observed that the result of the accuracy percentage in the test is even higher than in training. This tells us that the network has clearly learned to solve the problem; it has not overfitted and has not memorized connections between neurons.

4 Critical analysis

Thanks to Table 1, we observe that for this problem of 8×8 images it is not necessary to resort to multilayer networks and that other machine learning models are also suitable for its resolution. However, this is probably due to the relative simplicity of the problem to be solved. As soon as the problem requires greater complexity, such as higher quality images, people detection in video, or similar problems, neural networks optimize the task and infer the training examples better than machine learning techniques, especially the convolutional type.

If we try to solve the problem with the `MNIST` dataset and the neural network with one hidden layer, convergence is significantly slower and the necessary resources much greater (89% accuracy where the convolutional network achieved 97% with the same *epochs*). On the other hand, by increasing the number of layers and their neurons, the same accuracy rate has been achieved with the neural network. It would remain to contrast the computational cost of both trainings.

It is thus concluded that each problem requires a different algorithm, and that we must optimize our tools so as not to cause an overkill, or to avoid "using a sledgehammer to crack a nut" (such as using the convolutional network to recognize 8×8 images). But we should not fall into underfitting either, as would be the case of using the single-layer neural network for the `MNIST` dataset.